08-metadata/06/spec/spec_helper.rb
```
RSpec.configure do |config|
  config.define_derived_metadata(type: :model) do |meta|
    # ...
  end
end
```

Behind the scenes, RSpec uses the === operator to compare values for your filter expression. You'll hear more about this operator in *How Matchers Match Objects*, on page 176. For now, we'll just say that this style of comparison enables all kinds of things—like using a Ruby lambda in your filter expression. Most of the time, though, we find that regular expressions and exact matches are powerful enough.

Default Metadata

In the previous section, you used metadata to tag some of the examples automatically. Now, we'll do a twist on this concept: we're going to show you how to enable something by default on *all* of your examples, but allow individual examples to opt out.

Back in *Testing Ledger Behavior*, on page 84, you set the :aggregate_failures metadata on your integration specs to get more useful failure output. RSpec typically stops an example at the first failing expectation. With this tag in place, your integration specs would soldier on and report *every* failed expectation.

This aspect of RSpec is so useful that you may be tempted to enable it for every example:

08-metadata/07/aggregate_failures.rb
```
RSpec.configure do |config|
  config.define_derived_metadata do |meta|
    # Sets the flag unconditionally;
    # doesn't allow examples to opt out
    meta[:aggregate_failures] = true
  end
end
```

Switching on a feature globally can be extremely handy in situations like this. However, it's a good idea to let individual examples opt out of the behavior.

For instance, you might have some billing specs that hit a fake payment gateway. As an extra safety check, you define a before hook that stops any spec attempting to use the real gateway: